UC San Diego

Scalable Machine Learning – Spark

CIML Summer Institute 2026

Mai H. Nguyen, Ph.D.

SDSC - San Diego Supercomputer Center

SPARK

- Computing platform for scalable computing
  - Designed for big data workloads
  - Built-in parallelism & fault-tolerance on commodity cluster
  - Provides interactive querying, iterative analytics, streaming processing, along with batch processing
  - Goals: speed, ease of use, generality, unified platform
- History
  - Research project began in 2009 at UC Berkeley's AMPlab
  - Paper published in 2010
  - Contributed to Apache Software Foundation in 2013
  - Commercial version by Databricks

SPARK

- Goals: speed, ease of use, generality, unified platform
- In-memory processing
  - Exploits distributed memory to cache data
  - Intermediate results written to memory whenever possible

- How does Spark manage data in distributed system?

RESILIENT DISTRIBUTED DATASETS (RDDs)

- Spark central concept
  - Abstraction of data as distributed collection of objects

- Resilient Distributed Datasets (RDDs)
  - Data abstraction
  - Programming construct for storing and organizing data
  - Spark uses RDDs to distribute data and computations across nodes in cluster

RDD

- Resilient Distributed **Dataset**
  - Collection of data
    - From files in local filesystem (text, JSON, etc.)
    - From data store (HDFS, RDBMS, NoSQL, etc.)
    - Created from another RDD
- Resilient **Distributed** Dataset
  - Data is divided into partitions
  - Partitions are distributed across nodes in cluster
- **Resilient** Distributed Dataset
  - Provides resilience (e.g., fault tolerance) to failures
  - History of operations performed on each partition is tracked to provide lineage-based fault tolerance
- All provided automatically by Spark engine

SPARK CONTEXT

- Spark Context
  - Entry point to Spark engine
  - Provides way to create RDDs

```
from pyspark import SparkContext, SparkConf

conf = SparkConf()\
    .setAppName("RDD Example") \
    .config("config.option","config.value")
sc = SparkContext(conf=conf)
```

- SparkContext: connection to Spark engine
- SparkConf: configuration parameters for application

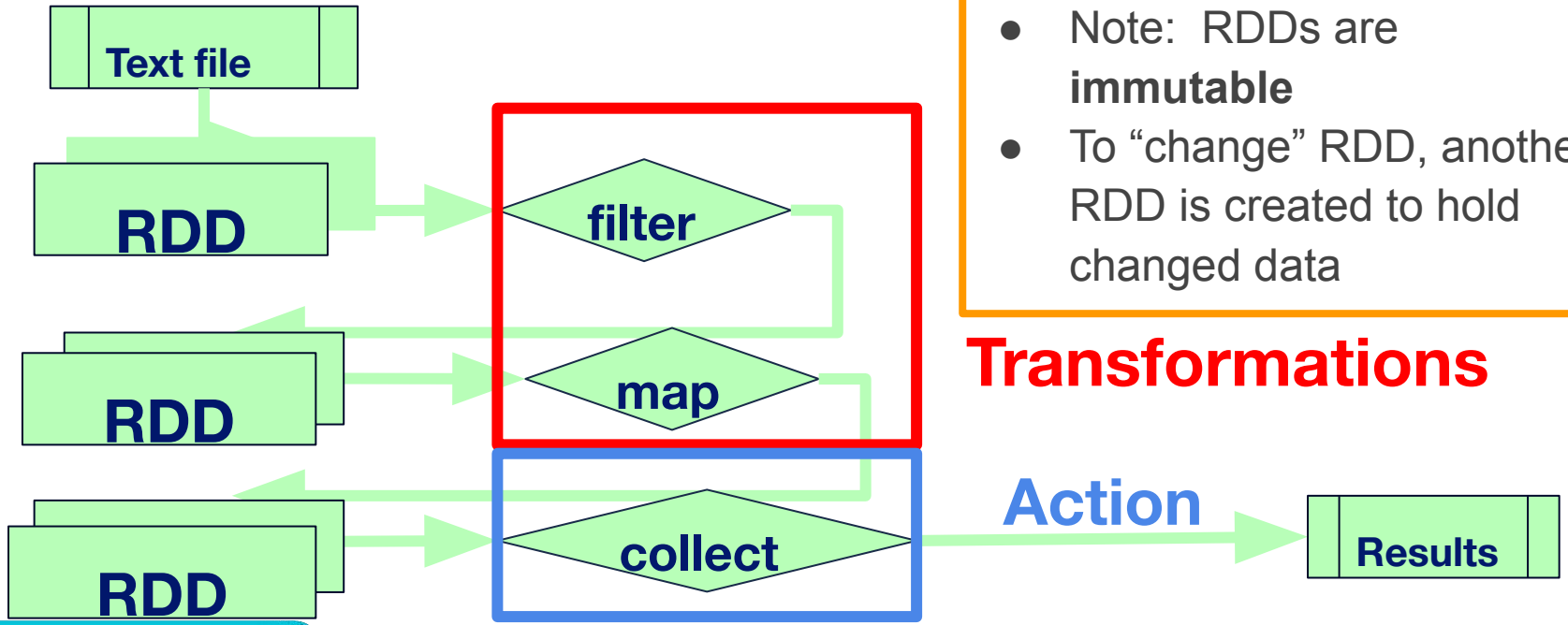
CREATING RDDs

- Read data from files in local filesystem (text, JSON, etc.)
  - lines = sc.textFile(“inputfile.txt”)
- Data read in from data store (HDFS, RDBMS, NoSQL, etc.)
  - lines = sc.textFile(“hdfs://<path>/inputfile.txt”)
- Generate data
  - numbers = sc.parallelize(range(100),3)
    - Divide data into 3 partitions
- Created by transforming another RDD
  - newLines = lines.filter(lambda s: “Spark” in s)

PROCESSING RDDs

UC San Diego

- RDDs can be processed using 2 types of operations
  - **Transformation**: Creates new RDD from existing RDD
  - **Action**: Runs computation(s) on RDD and returns value



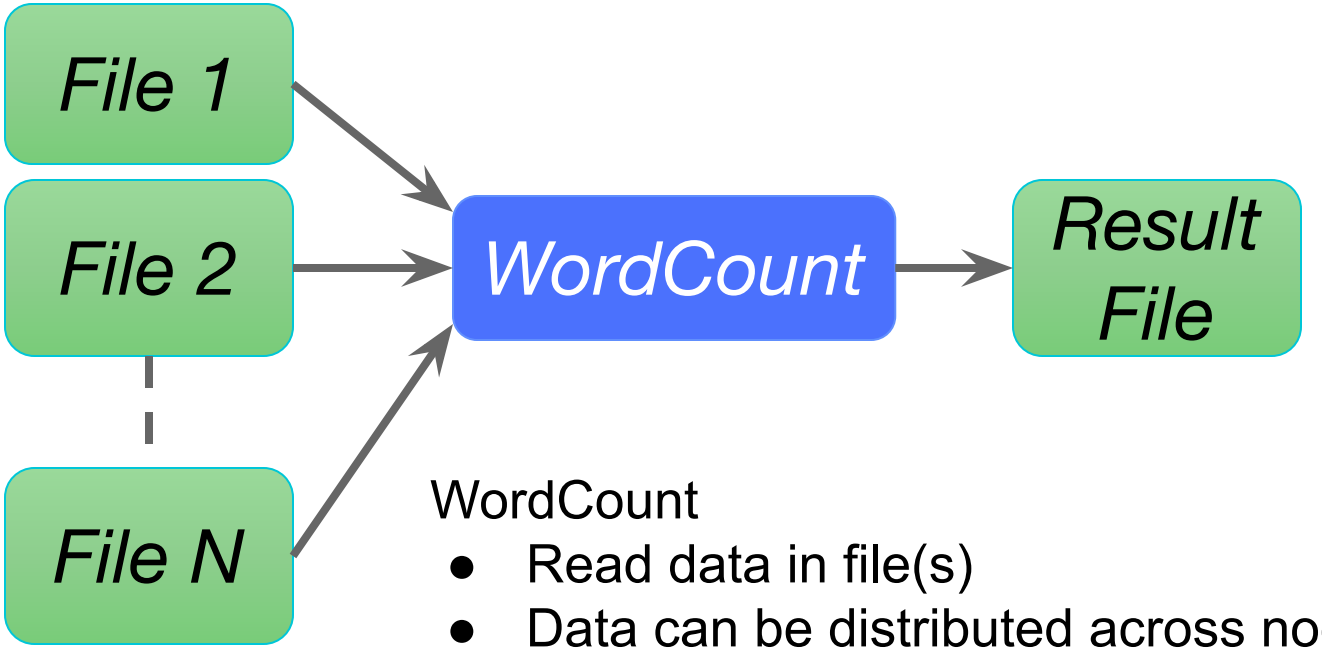
- Note: RDDs are **immutable**
- To "change" RDD, another RDD is created to hold changed data

Transformations

LAZY EVALUATION

- Transformations on RDDs have **lazy evaluation**
  - Transformations are not immediately processed
  - Plan of operations is built
- Operations executed when **action** is performed
  - i.e., actions force computation
- Allows for optimizations in generating physical plan
- Example:
  - filtered = strings.filter(strings.value.contains(“Spark”))
    - Nothing is returned
  - filtered.count()
    - ‘filter’ is performed, and count is returned

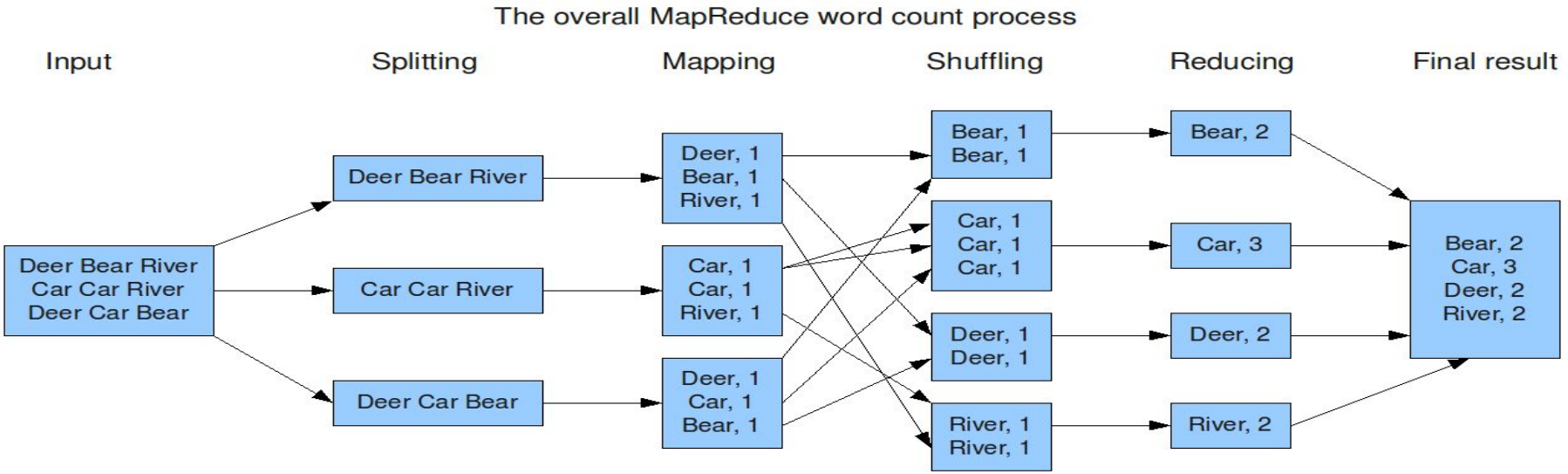
WordCount



WordCount

- Read data in file(s)
- Data can be distributed across nodes in cluster
- Count number of occurrences of each word

WordCount



https://www.todaysoftmag.com/article/1358/hadoop-mapreduce-deep-diving-and-tuning

Data is split into partitions

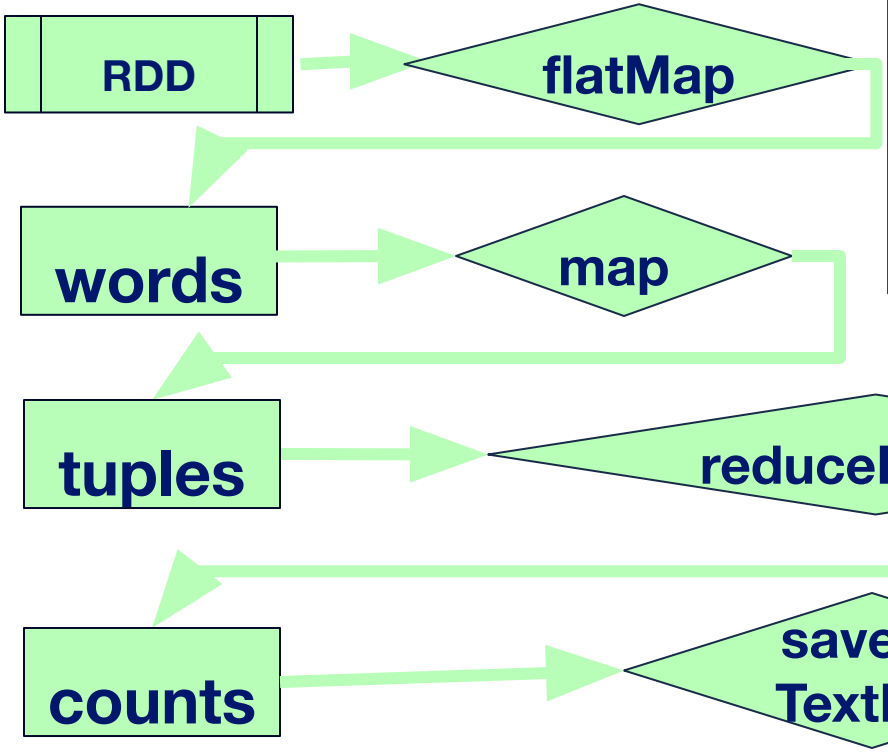
Map generates key-value pairs

Pairs with same key moved to same partition

Reduce sums values for each key

WordCount (RDD)

UC San Diego




```
text_file = sc.textFile("hdfs://..")
counts =
    text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word,1)) \
             .reduceByKey(lambda a,b: a+b)
counts.saveAsTextfile("hdfs://…")
```

DATAFRAMES & DATASETS

- Extensions to RDDs
  - Higher-level abstractions
  - Improved performance
  - Better scalability
- Can convert to/from RDDs and use with RDDs

DATAFRAMES & DATASETS

DataFrame

- Lazy evaluation
- Immutable
- Data organized as collection of Rows
- No static type checking
- APIs in Java, Scala, Python, R

DataSet

- Lazy evaluation
- Immutable
- Data organized as collection of Rows
- Provides static type checking
- APIs in Java and Scala

USING DATAFRAMES

UC San Diego

Spark Session

- Entry point to Spark engine
- Note that SparkContext is now **SparkSession**

```
from pyspark import SparkSession, SparkConf

conf = SparkConf \
   .setAll \
   ([("spark.app.name","DataFrame Example") \
   ("config.option","config.value")])

spark =
   SparkSession.builder.config(conf=conf)\
               .getOrCreate()
```

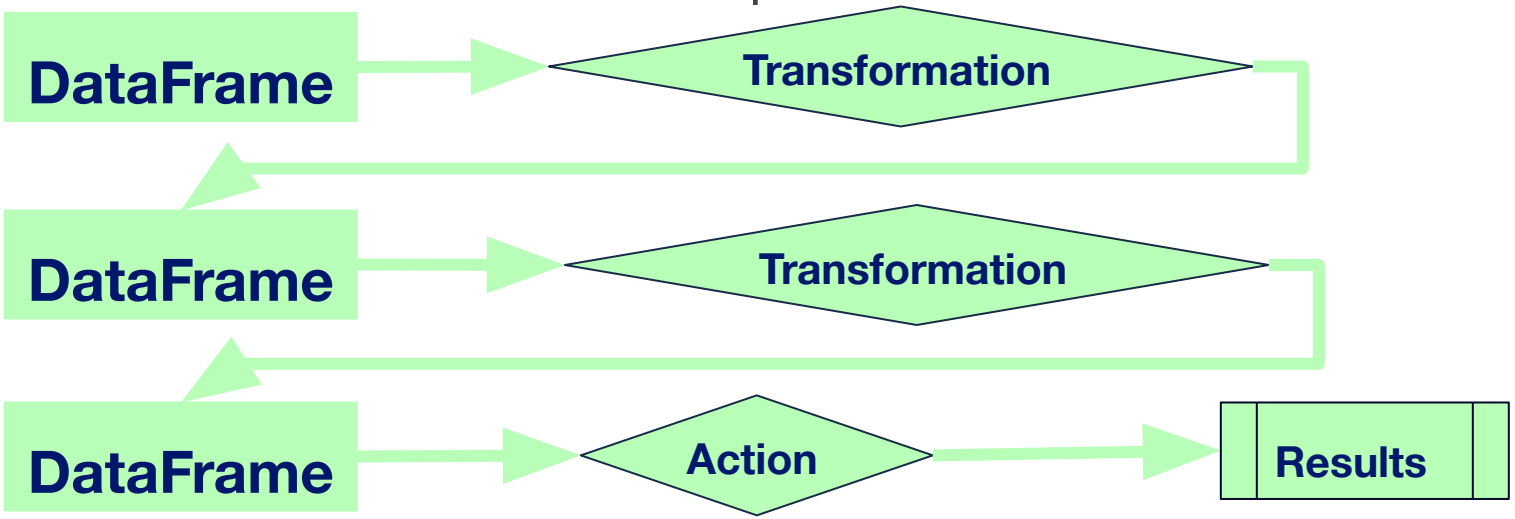
CREATING DATAFRAMES

- Read data from files in local filesystem (text, JSON, etc.)
  - df = spark.read.csv(“data.csv”, header=”True”)
- Data read in from data store (HDFS, RDBMS, NoSQL, etc.)
  - df = spark.read.csv(“hdfs:///<path>/data.csv”)
- Generate data
  - empl_0 = Row(id=”123”, name=”John”)
  - empl_1 = Row(id=”456”, name=”Mary”)
  - employees = [empl_0, empl_1]
  - df = spark.createDataFrame(employees)
- Created by transforming another DataFrame
  - filter_df = df.filter(col(“name”)==”Mary”))

DATAFRAME TRANSFORMATIONS & ACTIONS

UC San Diego

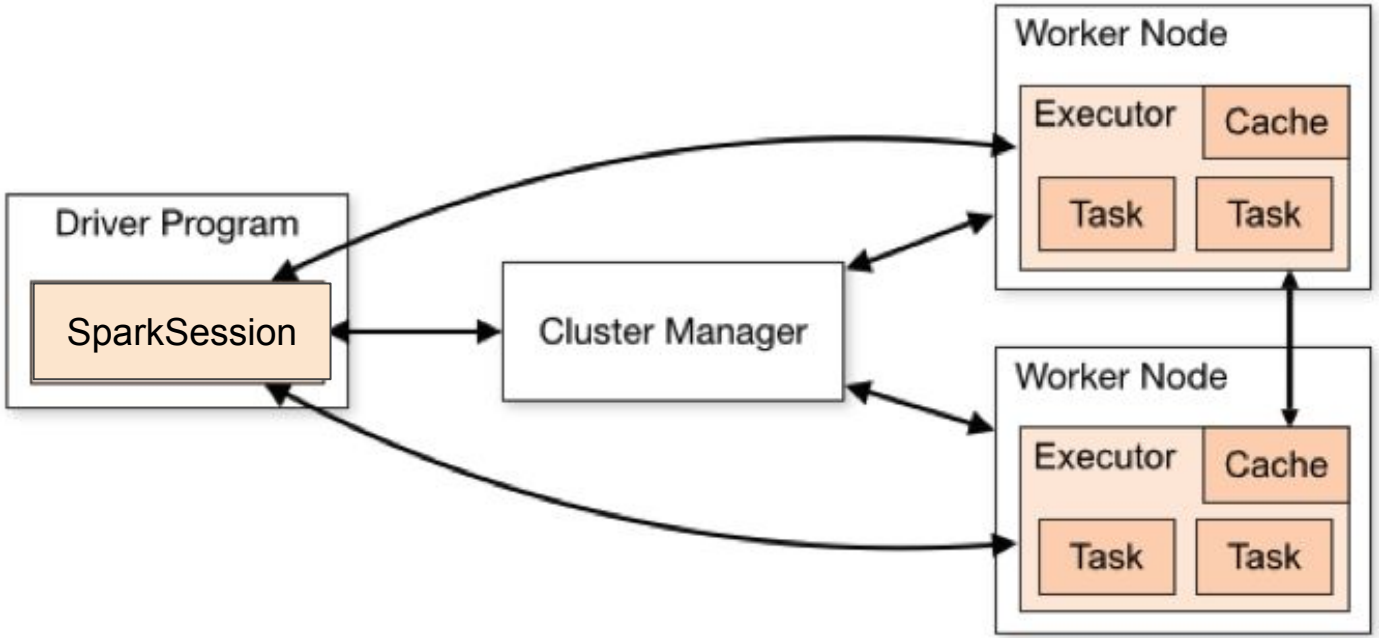
- Similar to RDDs, DataFrames can be processed using transformations and actions
- DataFrames are also immutable
- Transformations on DataFrames also have lazy evaluation
- Operations executed when action is performed



SPARK PROGRAM STRUCTURE

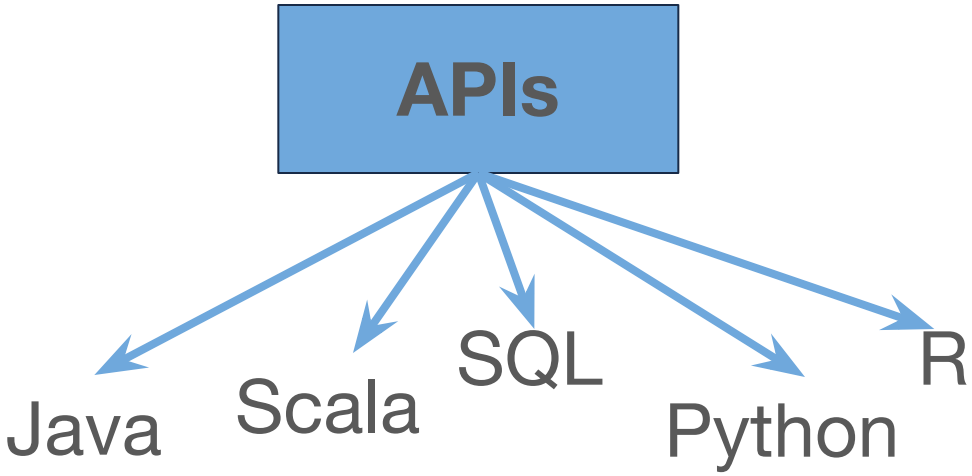
- Start Spark session
  - `spark = SparkSession.builder.config(conf=conf).getOrCreate()`
- Create distributed dataset
  - `df = spark.read.csv("data.csv",header="True")`
- Apply transformations
  - `new_df = df.filter(col("dept") == "Sales")`
- Perform actions
  - `df.collect()`
- Stop Spark session
  - `spark.stop()`

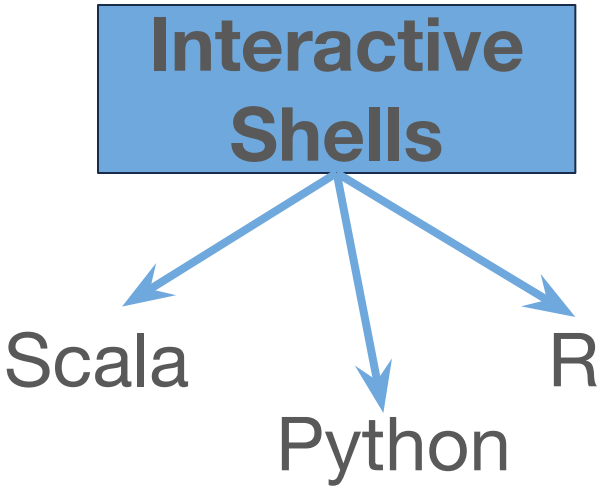
SPARK ARCHITECTURE



SPARK INTERFACE

Goals: speed, **ease of use**, generality, unified platform





RDD WORDCOUNT IN SPARK

Spark RDD API available in Python, Scala, Java, and R

```python
text_file = sc.textFile("hdfs://...")
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs://...")
```

```scala
val textFile = sc.textFile("hdfs://...")
val counts = textFile.flatMap(line => line.split(" "))
                 .map(word => (word, 1))
                 .reduceByKey(_ + _)
counts.saveAsTextFile("hdfs://...")
```

```java
JavaRDD<String> textFile = sc.textFile("hdfs://...");
JavaPairRDD<String, Integer> counts = textFile
    .flatMap(s -> Arrays.asList(s.split(" ")).iterator())
    .mapToPair(word -> new Tuple2<>(word, 1))
    .reduceByKey((a, b) -> a + b);
counts.saveAsTextFile("hdfs://...");
```

SPARK - GENERALITY

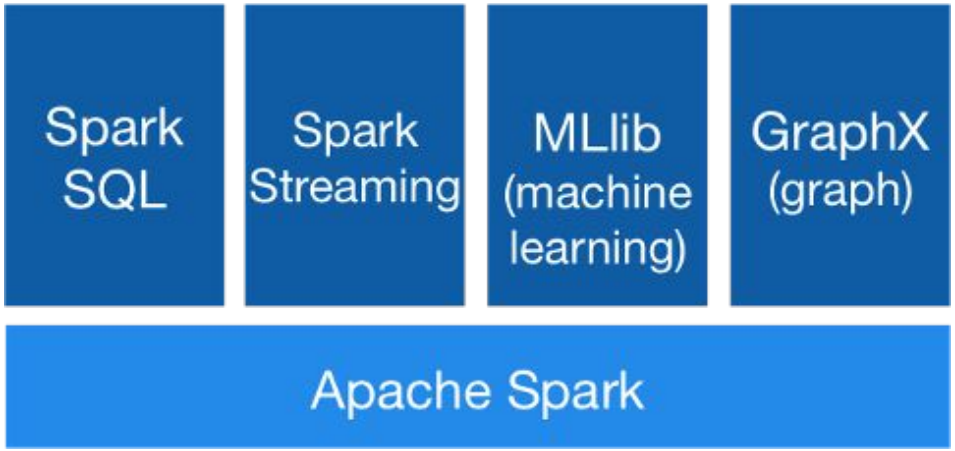
UC San Diego

- Goals: speed, ease of use, **generality,** unified platform
- Support for several data sources
  - Local file systems, HDFS, RDBMSs, MongoDB, Kafka, AWS S3, etc.
- Can run on various platforms
  - Hadoop, Kubernetes, cloud, standalone
- Support for multiple workloads
  - batch, streaming
  - machine learning, SQL, graph processing

SPARK – UNIFIED PLATFORM

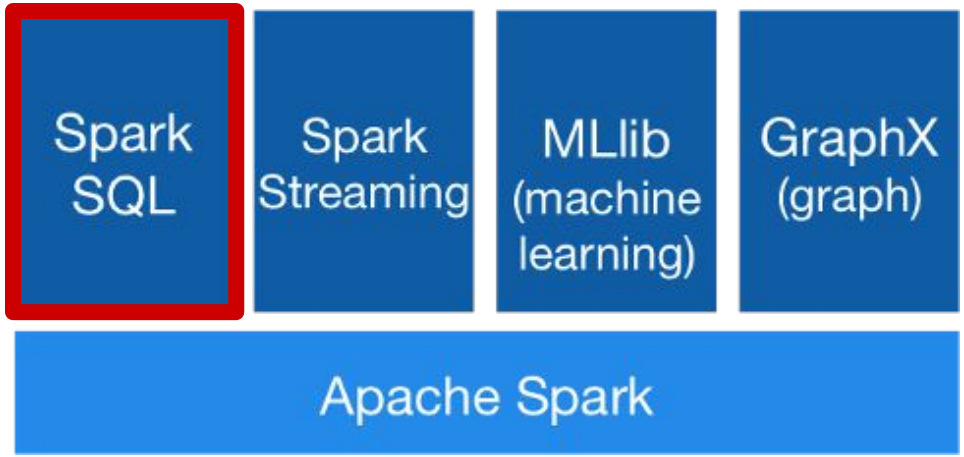
UC San Diego

Goals: speed, ease of use, generality, **unified platform**



- Provides unified platform for various analytics processing
- **Spark engine** provides core capabilities for scalable processing
- **Spark libraries** provide additional higher-level functionality for diverse workloads

SPARK SQL



- Structured Data Processing
  - Provides support for SQL and query processing
  - Has APIs for SQL, Scala, Java, Python, and R
  - Generated underlying code is identical

SPARK SQL

- Execute SQL queries
  - SQL

```
spark.sql("SELECT max(count)
FROM flight_data").take(1)
```

  - PySpark

```
from pyspark.sql.functions import max
flight_data.select(max("count")).take(1)
```

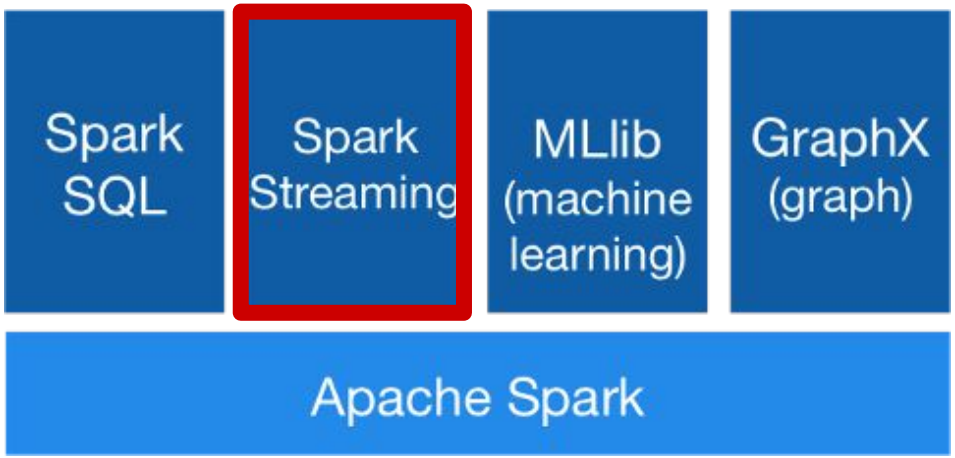
SPARK SQL

- Integrate SQL queries with Spark commands

```
df = spark.sql (“SELECT * FROM Employees”)
df.show(100)

num_employees =
   df.select(“Age”,”Dept”,”Salary”)
      .groupBy(“Dept”)
      .where(df.Salary > 80000)
      .count()
```

SPARK STREAMING



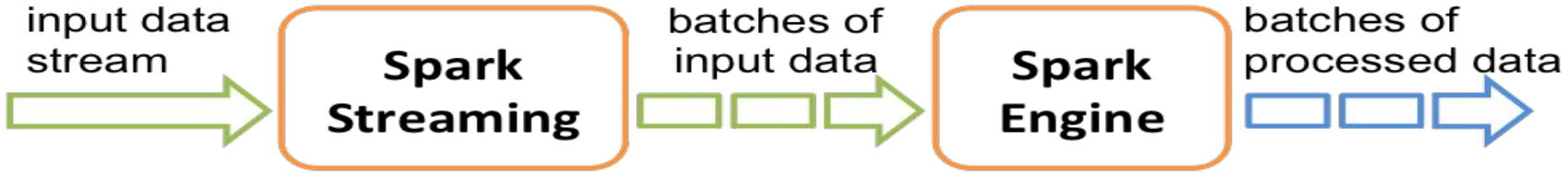
- Streaming Data Processing
  - Scalable processing for real-time analytics
  - Structured streaming
    - Data stream is divided into micro-batches of data
    - Same operations for static data can be used
  - Has APIs for Scala, Java, and Python

REAL-TIME ANALYTICS

- (Near) Real-Time Analytics
  - Analysis and use of data as it enters system
- Examples
  - Identifying fraudulent credit card transaction at point-of-sale
  - Viewing orders as they happen for up-to-date inventory tracking and trend analysis
  - Understanding trending topics of tweets/news articles/etc.

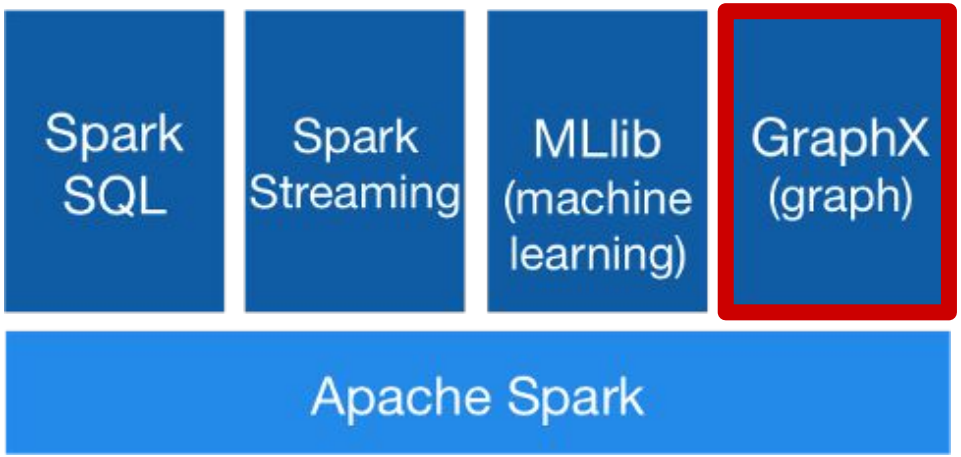
SPARK STREAMING

- Input data stream is divided into micro-batches of data that are processed by Spark engine
- DStream: high-level abstraction
  - Implemented as sequence of RDDs
- Any Spark operation can be applied to DStreams



https://spark.apache.org/docs/latest/streaming-programming-guide.html

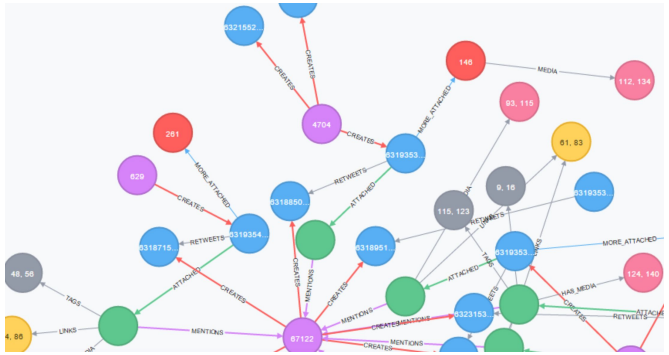
SPARK GRAPHX / GRAPHFRAMES



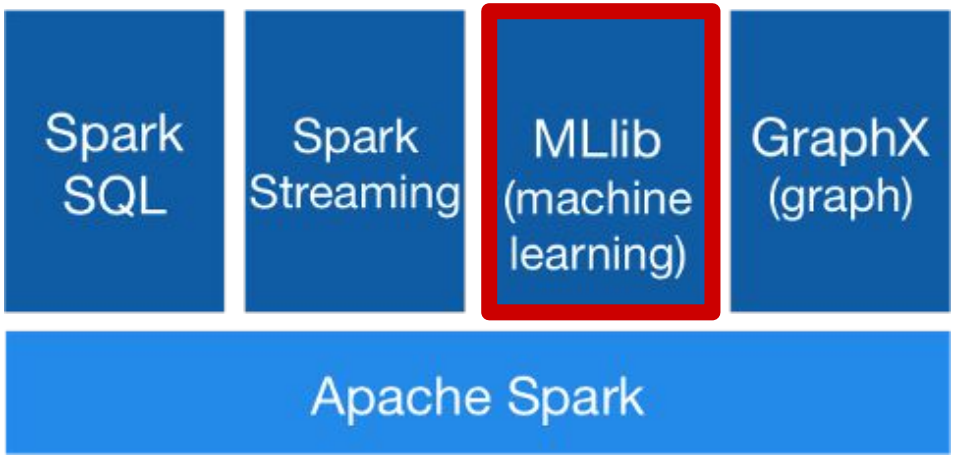
- Graph Computation
  - Scalable graph processing
  - Special structures for storing vertex and edge information & operations for manipulating graphs
  - GraphX (RDD-based) & GraphFrames (DF-based)
  - Has APIs in Scala, Java, Python (GraphFrames)

SPARK GRAPHX / GRAPHFRAMES

- Graph analytics
  - Analysis of relations among entities
- Data represented as graph
  - Entities are vertices
  - Relationships are edges
- Example: Analyzing tweets
  - Extract conversation threads
  - Find interacting groups
  - Find influencers in community



SPARK MLLIB



- Machine Learning
  - Scalable machine learning library
  - Scalable implementations of machine learning algorithms and utilities
  - Has APIs for Scala, Java, Python, and R

SPARK MLLIB ALGORITHMS

- Machine Learning
  - Classification, regression, clustering, etc.
  - Evaluation metrics
- Statistics
  - Summary statistics, sampling, etc.
- Utilities
  - Dimensionality reduction, transformation, etc.
- ML Pipelines
  - Similar to scikit-learn

MLLIB: STATISTICS

```
from pyspark.sql.functions import rand

# Generate random numbers
df = sqlContext.range(0,10)
       .withColumn("rand1", rand(seed=10))
       .withColumn("rand2", rand(seed=27))

# Show summary statistics
df.describe().show()

# Compute correlation
df.stat.corr("rand1","rand2")
```

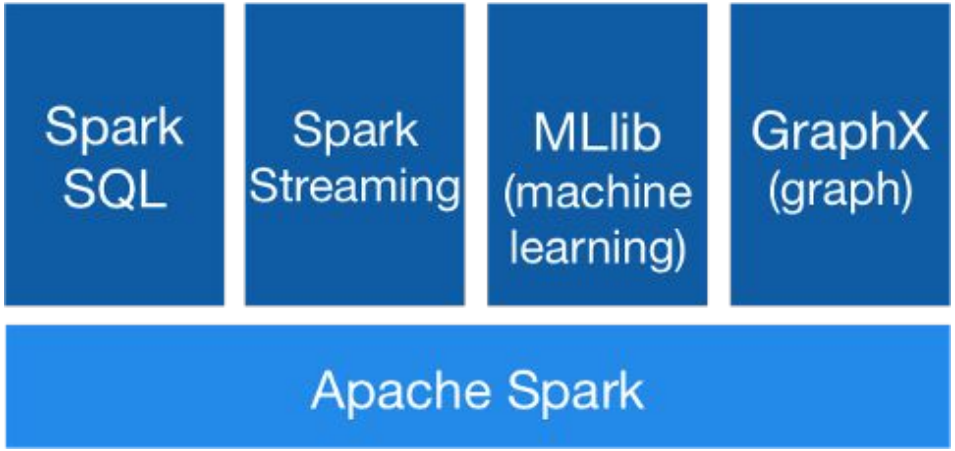
MLLIB: CLUSTER ANALYSIS

```
from pyspark.ml.clustering import KMeans

# Read and parse data
data = spark.read.csv("data.csv", header="true")


# k-means model for clustering
kmeans = Kmeans().setK(3).setSeed(123)
model  = kmeans.fit(data)
for centers in model.clusterCenters()
   print (centers)
```

SPARK LIBRARIES



- Spark Libraries
  - Use Spark engine as core
  - Extend functionality to particular applications
  - Libraries can be used together
  - Third-party packages: https://spark-packages.org

SPARK BEST PRACTICES

UC San Diego

- Pandas
  - Use Pandas if data fits into memory
- Start small
  - Work with data sample first to work through pipeline & debugging
- Be careful with actions such as collect() and take()
  - This collects all data to Driver. Will get OOM if insufficient memory.
- Cache data that is frequently accessed
  - e.g., data in training set for machine learning
  - Note that DF is not fully cached until action is invoked
- Use built-in functions instead of user-defined functions
  - Built-in functions are optimized

SPARK BEST PRACTICES

UC San Diego

- Load balance
  - Repartition data as needed
    - coalesce(N) reduces number of partitions without shuffling
    - repartition(N) increases/decreases number of partitions with shuffling
- Use right level of parallelism
  - Number of partitions should be >= number of cores for optimal resource optimization
  - Too many small partitions will require a lot of disk I/O
  - Too few large files will reduce parallelism

SPARK BEST PRACTICES

UC San Diego

- Lazy evaluation
  - Remember that transformations are performed only when action is executed
  - Can add action to force computations for debugging. But for efficiency, should have action only at end of pipeline.
  - Don’t execute cells in Jupyter notebook out of order
- Memory
  - Errors with Py4J are probably OOM-related
  - Can set driver memory, driver max result size, executor memory, etc. in Spark configuration:

```
conf = pyspark.SparkConf()
        .setAll([('spark.master', 'local[*]'),('spark.driver.memory', ‘4g’)])
```

DASK VS. SPARK

Dask

- Component in Python ecosystem
- Used in conjunction with numpy, pandas, scikit-learn
- Python-based
- Lacks high level optimization, but has flexibility to adapt to custom workloads
- Single node or cluster

Spark

- Big Data platform with its own ecosystem
- All-in-one project. Integrates with other Apache projects
- Has APIs in Scala, Java, Python, R, SQL
- Provides high level optimizations, but lacks flexibility for more complex or adhoc algorithms
- Single node or cluster

https://docs.dask.org/en/latest/spark.html

PANDAS API ON SPARK

UC San Diego

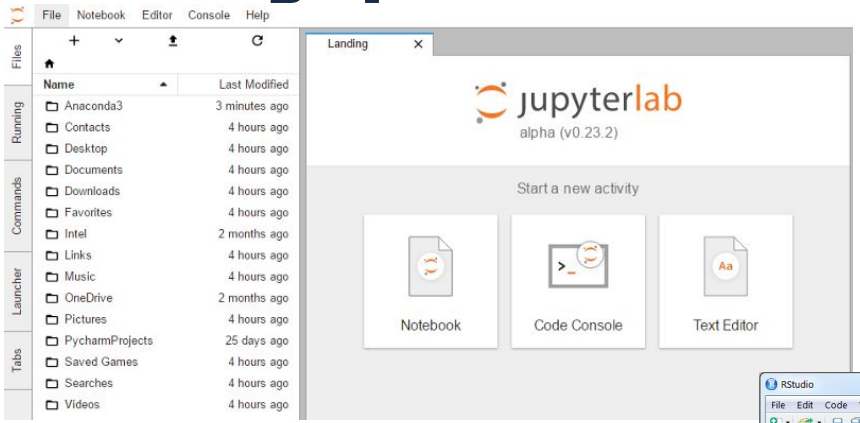
- Pandas API integrated into Spark as of v. 3.2
- Allows for simple porting of already existing Pandas code to Spark
  - Same code can run with pandas (single node) and Spark (distributed system)
- Can use Spark to scale to multiple machines
- Can take advantage of unified analytics provided by Spark
  - Can access functionality provided by other Spark libraries
- Covers subset of pandas API
- To use

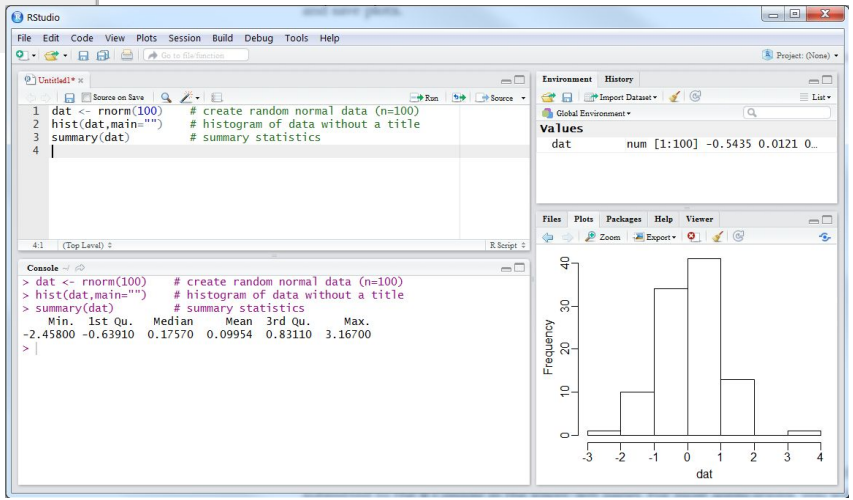
```
~~from pandas import read_csv~~
from pyspark.pandas import read_csv
pdf = read_csv("data.csv")
```

R FOR STATISTICAL ANALYSIS

- R
  - Open source programming language and software environment for statistical computing and graphics
  - Widely used for statistical analysis and machine learning
  - https://www.r-project.org/about.html
- RStudio
  - IDE (integrated development environment) for R
  - Has source code editor, build automation tools, debugger
  - https://www.rstudio.com/
- Powerful, but limited to single-machine execution

Running SparkR Code





sparklyr

RStudio

- R interface for Spark
- R Legacy
  - From Rstudio
  - Available on CRAN
- Spark backend to dplyr and SQL
  - Interactively manipulate Spark data using dplyr and SQL
- Access to Spark functionality
  - Interface to Spark MLlib algorithms
- Connect to Spark clusters

Spark Resources

- PySpark SQL Basics Cheat Sheet
  - PDF
- Spark Main Page
  - https://spark.apache.org/
- Spark Overview
  - https://spark.apache.org/docs/latest/index.html
- Spark Examples
  - https://spark.apache.org/examples.html
- Spark SQL, DataFrames and DataSets Programming Guide
  - https://spark.apache.org/docs/latest/sql-programming-guide.html
- Spark MLlib Programming Guide
  - https://spark.apache.org/docs/latest/ml-guide.html
- PySpark API Documentation
  - https://spark.apache.org/docs/latest/api/python/index.html

Setup

- Login to Expanse
  - Open terminal window on local machine
  - ssh login.expanse.sdsc.edu -l <account>
- Pull latest from repo
  - git pull
  - URL: https://github.com/ciml-org/ciml-summer-institute-2026.git

Server Setup for PySpark – Command Line

- In terminal window
  - jupyter-shared-spark
    - Alias for: galyleo launch --account <account> --partition shared --time-limit 4:00:00 --cpus 4 --memory 16 --conda-yml spark/conda-pyspark-3.5.5.yaml --mamba --cache

- To check queue
  - squeue -u $USER